

Course Outline: Database Normalization

Title: Database Normalization **Skill:** 40 — Cómo optimizar el almacenamiento para reporte, integridad, consulta o transporte **Learnpack:** + Normalización en bases de datos **File:** s40-w14d40-normalizacion-en-bases-de-datos **Prerequisite:** SQL Fundamentals (Skill 30), Managing Related Tables with SQL (Skill 31) **Target Audience:** Beginner developers who understand SQL basics and want to design better database schemas **Total Lessons:** 9 **Format:** Conceptual (0% hands-on)

Course Description

Bad database design creates problems that grow over time: duplicate data, inconsistent records, and queries that collapse under load. This course teaches database normalization — the discipline of organizing data to eliminate redundancy and preserve integrity. Students learn the two normal forms that matter most in practice (1NF and 3NF), how to choose between them, and — critically — when to intentionally break the rules for performance.

Course Philosophy

This course emphasizes:

- Design decisions over syntax: normalization is about thinking, not typing
 - Trade-offs over dogma: 3NF is the standard, but real systems break it deliberately
 - Write vs. read workloads as the lens for all decisions
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Normal Forms	3
02 - Applying Normalization	3
03 - Assessment	1
04 - Outro	1
Total	9

00.0 Welcome to Database Normalization

Learning Objectives:

- Understand what problem normalization solves and why it matters
- Preview what the course covers and what you will be able to do by the end

Content Outline:

- The real cost of bad database design: a relatable scenario (duplicate emails, inconsistent city names, orphaned records)
- What normalization is and what it is not
- Course overview: normal forms, choosing between them, and when to break the rules

Transitions: This lesson motivates the course. Section 01 starts immediately with what normal forms are and why they emerged as a solution.

Key Principles:

- Normalization is a discipline for reducing data redundancy and preserving integrity
- The two forms that matter most in practice are 1NF and 3NF — the rest are academic

Examples:

- A **users** table storing city, country, and postal code as free text — and the chaos that follows when users spell "New York" five different ways
 - An **orders** table duplicating customer names — and what happens when a customer changes their name
-

01.0 Normal Forms

Learning Objectives:

- Explain what a "normal form" is and why the concept exists
- Identify the two normal forms this course covers and how they relate to each other
- Describe the progression: unnormalized → 1NF → 3NF

Content Outline:

- What is a normal form? (a set of rules a table must satisfy)
- Where normal forms came from: Edgar Codd's relational model, 1970
- The idea of "levels": each higher form builds on the previous
- Why we skip 2NF in practice: 1NF and 3NF are the decision points that matter
- Overview of what 01.1 and 01.2 will each cover

Transitions: This lesson frames the concept. Lesson 01.1 dives into the specific rules of First Normal Form with concrete before-and-after examples.

Key Principles:

- A normal form is a contract: a table either satisfies it or it doesn't
- Higher forms are stricter — a table in 3NF is also in 2NF and 1NF
- The goal is always the same: reduce redundancy, preserve integrity

Examples:

- A spectrum diagram: unnormalized table → 1NF → 2NF → 3NF, with 2NF greyed out as "skipped in practice"
 - Before: a `contacts` table with a `phones` column containing "555-1234, 555-5678" — violating 1NF
 - After: a separate `contact_phones` table — satisfying 1NF
-

01.1 First Normal Form

Learning Objectives:

- State the rules of First Normal Form
- Identify violations of 1NF in a given table
- Restructure a table to satisfy 1NF

Content Outline:

- The two rules of 1NF: atomic values, no repeating groups
- What "atomic" means: each cell holds exactly one value
- What "repeating groups" means: no arrays, no comma-separated lists, no numbered columns (phone1, phone2, phone3)
- How to fix a 1NF violation: extract the repeating group into a new table
- When 1NF violations are tempting and what goes wrong

Transitions: 1NF eliminates multi-valued cells. Lesson 01.2 goes further: 3NF eliminates a subtler problem — columns that depend on each other rather than on the primary key.

Key Principles:

- Every column must hold one value — not a list, not a JSON blob, not a delimited string
- Repeating groups (phone1, phone2, phone3) are a 1NF violation hiding behind separate columns
- Violating 1NF makes queries painful: you cannot filter, sort, or join on half a cell

Examples:

- Violation: `orders(id, product_ids)` where `product_ids = "1,4,7"` — to find all orders containing product 4, you must parse strings
 - Fix: `order_items(order_id, product_id)` — a join table that makes the query trivial
 - Anti-pattern: using JSON columns in a relational database to avoid designing a proper schema — it defers the pain, it does not remove it
-

01.2 Third Normal Form

Learning Objectives:

- State what Third Normal Form requires
- Identify transitive dependencies in a table
- Restructure a table to satisfy 3NF

Content Outline:

- What 3NF adds beyond 1NF: eliminating transitive dependencies
- What a transitive dependency is: a non-key column that depends on another non-key column, not on the primary key
- The test: "Does this column describe the row, or does it describe another column?"
- How to fix a transitive dependency: extract the dependent columns into their own table
- The payoff: updates in one place propagate everywhere automatically

Transitions: Now you know what both forms require. Lesson 02.1 compares them directly and gives you a decision framework for which to apply.

Key Principles:

- In 3NF, every non-key column describes the primary key — nothing else
- Transitive dependencies create update anomalies: change a city's zip code and you must update every row that mentions that city
- 3NF is the default for write-heavy, integrity-critical data

Examples:

- Table: `employees(id, name, department_id, department_name, department_budget)` — `department_name` and `department_budget` depend on `department_id`, not on `id`
 - Fix: `departments(id, name, budget) + employees(id, name, department_id)` — updating a department's budget now requires changing exactly one row
 - Real-world consequence of skipping 3NF: a customer's city is stored in 10,000 order rows; the customer moves; 10,000 rows need updating; one missed → data is permanently inconsistent
-

02.0 Applying Normalization

Learning Objectives:

- Describe the decision framework for choosing between normal forms
- Explain why normalization is a default, not an absolute
- Identify the two dimensions that drive the decision: write vs. read workloads

Content Outline:

- Normalization is a default: 3NF unless there is a reason not to
- The two workload types and why they favor different designs
 - Write-heavy (transactional): integrity is paramount → normalize
 - Read-heavy (reporting, analytics): speed is paramount → sometimes denormalize
- Overview of what 02.1 and 02.2 will cover

Transitions: This section overview establishes the decision mindset. Lesson 02.1 walks through the specific criteria for choosing between 1NF and 3NF; lesson 02.2 covers when to deliberately step back from full normalization.

Key Principles:

- Every normalization decision is a trade-off between integrity and query performance
- The workload type — not personal preference — drives the decision
- Breaking normalization rules is valid only when done deliberately, with understanding of the consequences

Examples:

- A user action (clicking "enroll" in a course platform) → write → stored in 3NF
 - A weekly retention report across 10M user events → read → often served from a denormalized summary table
-

02.1 Choosing Between 1NF and 3NF

Learning Objectives:

- Apply a decision framework to determine which normal form a table should satisfy
- Explain why 3NF is the default for transactional data
- Identify the specific scenario where stopping at 1NF is appropriate

Content Outline:

- The decision question: "Will this data be written often, or mostly read?"
- User actions as the canonical write workload: enrollments, clicks, purchases, form submissions — constant writes that must be consistent
- Why 3NF protects write-heavy data: one source of truth, no update anomalies, referential integrity
- When 1NF is enough: log-style tables where rows are immutable and append-only (event logs, audit trails)
- The 1NF vs 3NF comparison: what each form eliminates, what it costs, where it pays off

Transitions: You now know when to normalize fully. Lesson 02.2 covers the deliberate move in the other direction: denormalization for performance.

Key Principles:

- 3NF is the default for any data that gets updated after it is written
- Constant user writes (actions and their consequences) belong in 3NF: no redundancy, guaranteed integrity
- 1NF is appropriate for immutable log data where transitive dependencies never cause update anomalies (the row never changes, so it cannot become inconsistent)

Examples:

- `user_enrollments(id, user_id, course_id, enrolled_at)` — 3NF, updated if a user unenrolls; wrong to store `course_name` here because it would go stale
- `click_events(id, user_id, element, page, timestamp)` — append-only log; transitive dependency between `element` and `page` is harmless because the row never changes
- Decision checklist: Does this row ever change after insert? Yes → 3NF. Never → 1NF may be sufficient.

02.2 When to Denormalize

Learning Objectives:

- Explain why relational databases are not typically used directly for reports
- Describe denormalization and the specific performance problem it solves
- Identify when denormalization is appropriate and what it costs

Content Outline:

- Why relational databases are optimized for writes, not reads
- The performance problem with fully normalized data at report scale: joins across millions of rows are slow
- What denormalization is: deliberately introducing redundancy to eliminate joins
- Denormalized reporting tables: pre-aggregated, pre-joined, updated on a schedule or by event
- The cost of denormalization: redundancy, potential staleness, more complex write logic
- The rule: never denormalize the source of truth — denormalize a copy

Transitions: This is the final lesson before the assessment. You now have the complete picture: normalize your transactional data, denormalize your reporting layer when performance demands it.

Key Principles:

- Relational databases are write-optimized; complex analytical queries are typically not their strength at scale
- Denormalization is a deliberate trade: you accept redundancy in exchange for read speed
- The source of truth stays normalized — only the reporting layer is denormalized
- Staleness is the risk: denormalized tables must be refreshed, and the refresh window defines how "live" the data is

Examples:

- Normalized source: `orders` joined to `order_items` joined to `products` joined to `customers` → a weekly sales report requires 4 joins across millions of rows
- Denormalized report table: `weekly_sales_summary(week, product_id, product_name, category, total_revenue, units_sold)` — pre-computed nightly, one scan for the report
- Real pattern: OLTP database (normalized, 3NF) → nightly ETL → data warehouse (denormalized, columnar) → BI tools query the warehouse, not the OLTP database

03.0 Normalization in Practice

Learning Objectives:

- Apply knowledge of 1NF, 3NF, and denormalization to realistic database design scenarios
- Identify violations and justify corrections
- Evaluate design trade-offs based on workload type

Question Format: Scenario-based

Questions:

1. A table `events(id, user_id, tags)` stores tags as a comma-separated string: `"python,backend,sql"`. Which normal form does this violate, and what is the correct fix?
2. A table `orders(id, customer_id, customer_name, customer_city, product_id, product_name, unit_price)` is in production. A customer moves to a different city. What problem does this table's design create, and which normal form rule does it violate?
3. Your team is designing a table to store every button click on a web application. Each row is written once and never updated. Your senior engineer suggests not worrying about transitive dependencies for this table. Is she right? Explain why or why not.
4. You are building a monthly revenue report that joins `orders`, `order_items`, `products`, `customers`, and `regions`. The query takes 45 seconds on production data. A colleague suggests creating a `monthly_revenue_summary` table populated nightly. What is this pattern called, and what is its main trade-off?
5. A table `employees(id, name, zip_code, city, state)` stores address data. The `city` and `state` columns are determined by `zip_code`, not by `id`. What is this called, and which normal form fixes it?
6. A startup stores all user activity in a single table: `activity(id, user_id, action, metadata)` where `metadata` is a JSON blob containing different fields depending on the action type. This satisfies 1NF on the `metadata` column. True or false — and explain.
7. Your database powers a booking system. Reservations are created and sometimes cancelled or modified. A teammate proposes storing `hotel_name` and `hotel_star_rating` directly in the `reservations` table for convenience. What is the risk, and what is the normalized alternative?
8. You are told: "Relational databases are write-optimized, so we use a separate analytical database for reports." Explain what this means in terms of normalization and workload types.

Evaluation Criteria:

- Correctly identifies which normal form applies to each scenario
- Justifies decisions with reference to workload type (write vs. read)
- Recognizes denormalization as a deliberate choice, not a mistake
- Does not conflate 1NF and 3NF violations

Score Interpretation:

- 7-8 correct: Ready to design production schemas
- 5-6 correct: Solid foundation; review the scenarios you missed
- Below 5: Re-read sections 01 and 02 before moving on

04.0 Conclusion

Content Outline:

- Course recap: what students accomplished
 - Learned what normal forms are and why they were developed
 - Understood the rules of 1NF and 3NF with concrete examples
 - Built a decision framework: 3NF as the default, 1NF for immutable logs
 - Understood denormalization as a deliberate performance trade-off
 - Connected database design to workload types: writes need integrity, reads need speed
- Connection to bigger picture
 - These decisions show up every time you design a schema, build a migration, or debug a slow query
 - The next skills build on this: telemetry data (high-volume writes) and reporting (read-optimized summaries) are exactly the workloads covered here
- Next steps and continued learning
 - Practice: take an existing schema you know and apply the 3NF test to each table
 - Explore: PostgreSQL's EXPLAIN ANALYZE — see the cost of joins on unnormalized data
 - Read: Martin Kleppmann's "Designing Data-Intensive Applications" ch. 2 for a deeper treatment of data models
- Encouragement
 - Good database design is invisible — nobody notices when it works. You now have the tools to make that happen.

Note: This is conclusion only — no theory, exercises, or assessment.